Example: $\sum_{n \geq 1} \frac{1}{n^p}$ has the same nature as $\int_1^\infty \frac{1}{x^p} dx$. (by the integral test)

Curs 1

19.01.2026 - 8.02.2026

↳ 2 dates for the written exam

$\begin{cases} 21.01.2026 \\ ? \quad 26-30.01.2026 \end{cases}$

↳ marire: 16.02.2026 - 22.02.2026
+ restanțe 16.02.2026

Location Counter

segment data...

a db 1, -2, 0FFh, 'xyz'

db...

db...

label db $-a ; correct pointer arithmetic ✓

||| beginning of the curr. seg. ; mov [label],... ✓

label dw $-$$ ✓

label equ $-a ✓ ; mov [label],...

↳ label is a constant

label dw $-data ✗

label dw label-a ✓ ; label equ label-a

times ✗ label dw 'a' ✗ ; times label dw 'a'

b equ 27 ; b is not an offset

c dd 12345678h

label dw b - a ✗

(b: const., a: pointer)

label dw c-a ✓

label dw c-a-4 ✓

label dd $ - a ✓

CS, DS, SS, ES, FS, GS → segment register

mov eax, [v] ✓ ; mov eax, dword ptr ds:[405000]

mov eax, [ebx] ; mov eax, dword ptr ds:[ebx]

offset: [base] + [index * scale] + [constant]

mov eax, [ebp*2] ; mov eax, dword ptr ss:[ebp+ebp]

mov eax, [ebp*3] ; — " — ss:[ebp+ebp*2]

mov eax, [ebp*4] ; — " — ds:[ebp*4]

mov eax, [ebp + esp] ; dword ptr [SS: esp+ebp]

(ebp: index, esp: base)

mov eax, [esp+ebp] ; — " —

mov eax, [ebx + esp*4] ; syntax error

esp!!! → cannot be index

mov eax, [ebx + ebx*4] ; ... DS:[..]..

(data seg.)

mov eax, [ebx + ebp] ; DS:

mov eax, [ebp + ebx] ; SS

(stack seg.)

mov eax, [ebx*1 + ebp*1] ; SS.

(ebx: index, ebp: base)

Bitwise Operations and Instructions

→ difference between operator and instructions

```
mov al, 01110111b <<3      (assembly time)
        ;al=10111000b
or
mov al, 01110111b
                           (run time)
shl al, 3
        ; al = 10111000
```

→ AND, OR, XOR, NOT

x - one bit

x and 0 ⇒ 0	x and x ⇒ x
x and 1 ⇒ x	x and ~x ⇒ 0
x or 0 ⇒ x	x or x ⇒ x
x or 1 ⇒ 1	x or ~x ⇒ 1
x xor 0 ⇒ x	x xor x ⇒ 0
x xor 1 ⇒ ~x	x xor ~x ⇒ 1

→ Different ways to write 0 in eax.

```
mov eax, 0
```

1. xor eax, eax
2. and eax, 0
3. shl eax, 32 ; shr, sal
4. sub eax, eax
5. mul [redacted] dword 0

→ operators ! and ~ (1's complement)

logic negation

- in C language: 0 false, everything else true (we prefer 1)
- in ASM, same as C !x=0 when x≠0, otherwise 1

mov al, ~0 ; AL = 0FFh

data segment

a d?

b d?

mov eax, ![a] X ; syntax error
; a is not det. at ass. time

after !, we need ass. time

mov eax, [!a] X - 4 -

↳ applied only to constants,
not to pointers

mov eax, !a ; X appl. only to scalars

mov eax, ! (b-a) ✓

↳ value, no constant

↳ pointer - pointer = value,
not pointer

mov eax, !f ; eax = 0

mov eax, !0 ; eax = 1

mov eax, ~f ; eax = 0F8h

f = 0000 0111b

~f = 1111 1000
F 8

mov eax, !ebx ; X not ass. time

constant OK

(aa equ 2)

mov ah, !aa ; ✓ AH = 0

mov ah, 1f ^ (~1f) ; AH = 0FFh

Type operators and operands data type

↳ perform computations only with constant values, with one exception "+" for offset: eg: [eax + ebx + 2]

v d?

a d?

b d?

✓ push v ; stack ← offset v

push [v] ; syntax error ✗ ; we don't know the size ; it gets the value, instead of the offset

push dword [v] ✓

push word [v] ✓

mov eax, [v] ✓ ; eax, dword ptr ds:[v]

push [eax] ✗

push 15 ; ⇐> push dword 15 (bits 32)

push dword / word [v] ✓

pop v ✗ (R)

a = 2 R

[pop [v] ???

pop dword [v]

mov [v], 0 ; operand size not specified

?bits

mov byte [v], 0

Error types

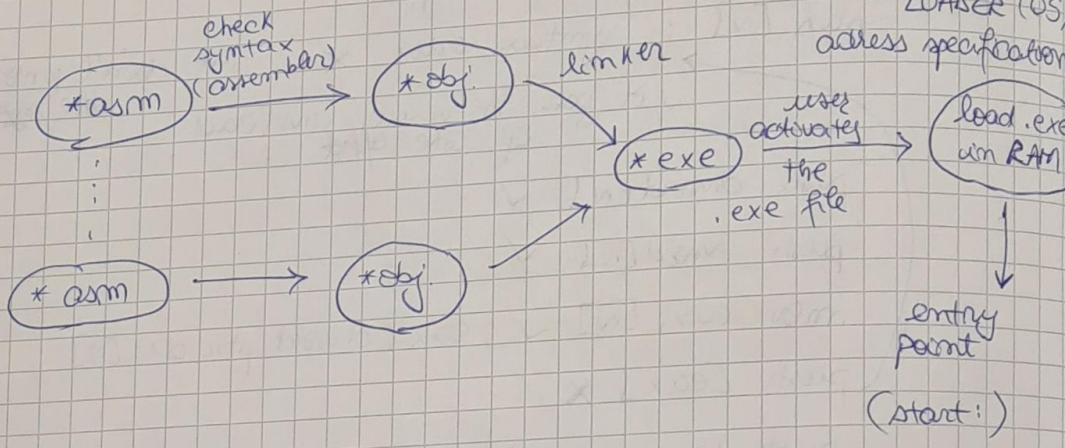
push 15

→ syntax error ⇒ generated by the assembler

→ run-time error (execution error) = program crashes...

→ logical error = program runs until the end or it can remain blocked in an infinite loop (x>1 and x<0) ??

→ linking errors = same variable declared in multiple *asm. files



data seg..

a1 db 0,1,2,'xyz'

00	01	02	'x'	'y'	'z'

↑ a1

a2 db ~~300~~ "F" + 3

46

49

↑ a2

46h = ASCII of 'F'?

a3 times 3 db 44h

44	44	44

↑ a3

↑ I reserve 3 bytes

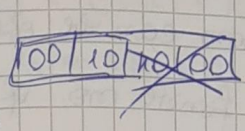
a4 db a8+1 ; syntax error

a5 dw a2+1 ; ✓

↳ I can only take the 16 first bits, so word is valid, but byte is not

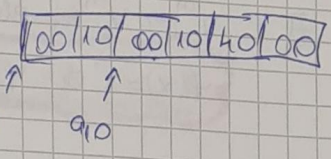
b dd 00401000h [b] = 00401000₈

a9 dw b

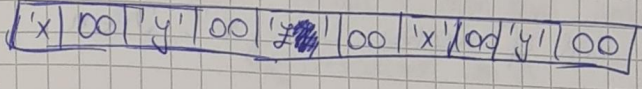


a9

a10 dd b



a11 dw 'xyz', 'x', 'y', 'z'



ALG

Curs 7

Elementary Operations

Let $_K V$. Then an elementary operation is one of the functions $e_{ij}, e_{i\alpha}, e_{ij\alpha} : V^m \to V^m$ defined for every $(v_1, \dots, v_m) \in V^m$ by: ($V^m = V \times V \times \dots \times V$)

$$e_{ij}(v_1, \dots, v_i, \dots, v_j, \dots, v_m) = (v_1, \dots, v_j, \dots, v_i, \dots, v_m)$$

~ interchanges v_i and v_j ~

$$e_{i\alpha}(v_1, \dots, v_i, \dots, v_m) = (v_1, \dots, \alpha \cdot v_i, \dots, v_m) \; ; \alpha \in K^*$$

$$e_{ij\alpha}(v_1, \dots, v_i, \dots, v_j, \dots, v_m) = (v_1, \dots, v_i + \alpha v_j, \dots, v_j, \dots, v_m) \; ; \alpha \in K$$